Lab 2 Worksheet Assignment Report

Training a CNN on CIFAR-10 with FLOPs, Gradient Flow, and Weight Update Analysis

Course: ML-DL-Ops

Submitted By: Sushantak Parashar Jha

Roll No: M25CSA035

Program: M.Tech AI

---

1. Objective

The objective of this experiment is to design and train a custom Convolutional Neural Network (CNN) on the CIFAR-10 dataset, analyze its computational complexity using FLOPs, and study the training dynamics through gradient flow and weight update flow visualizations.

All experiments and visualizations are logged using Weights & Biases (WandB) for reproducibility and analysis.

2. Dataset

The CIFAR-10 dataset consists of:

- 60,000 RGB images of size 32 × 32.
- 10 object classes.
- 50,000 training images and 10,000 test images.

A custom PyTorch Dataset and DataLoader were implemented. The training set was further split into training and validation subsets. Standard data augmentation techniques, such as random horizontal flipping and random cropping, were applied to improve generalization.

3. Model Architecture

A custom CNN architecture was designed specifically for CIFAR-10 without using any pretrained models. Key architectural components include:

- **Convolutional Blocks:** Multiple blocks with 3×3 kernels for feature extraction.
- **Batch Normalization:** Applied to stabilize training and improve convergence speed.
- **ReLU Activations:** Used for non-linearity.
- **Max Pooling:** Used for spatial downsampling.
- **Classifier:** A fully connected head with Dropout to reduce overfitting.

The model progressively increases the number of feature maps while reducing spatial resolution, enabling hierarchical feature learning.

4. Computational Complexity (FLOPs Analysis)

The computational complexity of the model was analyzed using the ptflops library.

- **FLOPs:** ~40 million operations
- **Number of Parameters:** ~1.3 million

This indicates that the model is computationally efficient while remaining expressive enough for CIFAR-10 classification.

5. Training Setup

The model was trained on a CPU environment with the following configuration:

- **Optimizer:** Stochastic Gradient Descent (SGD) with momentum.
- **Learning Rate:** 0.01
- **Batch Size:** 128
- **Loss Function:** Cross-Entropy Loss
- **Epochs:** 30

No trained model weights were saved, as per assignment instructions.

6. Training and Validation Performance

At the end of the 30-epoch training cycle, the model achieved the following metrics:

- **Training Accuracy:** ~99%
- **Training Loss:** 0.03

- **Validation Accuracy:** ~84.3%`
- **Validation Loss:** ~0.78

The training loss decreased steadily across epochs, indicating effective optimization. The validation accuracy improved rapidly during early epochs and then stabilized, while the validation loss showed mild fluctuations in later epochs. This behavior suggests limited overfitting and reasonable generalization performance for a model trained from scratch. All metrics were logged and visualized using **Weights & Biases (WandB)**.

7. Gradient Flow Analysis

Gradient flow visualizations were generated to study how gradients propagate across different layers during training.

- **Distribution:** Gradients were well-distributed across all layers.
- **Stability:** No evidence of vanishing or exploding gradients was observed.
- **Impact:** Batch Normalization helped maintain stable gradient magnitudes.

These observations confirm that the network trains in a stable and well-conditioned manner.

8. Weight Update Flow Analysis

Weight update flow visualizations were used to analyze how much each layer's weights change over time.

- **Early Epochs:** Larger weight updates were observed as the model learned initial features.
- **Convergence:** Update magnitudes gradually decreased as training progressed.
- **Deep Layers:** Later layers showed smaller updates in the final epochs, indicating convergence.

9. Experiment Tracking (WandB)

All experiments were logged using Weights & Biases, including training/validation loss, accuracy curves, and flow visualizations.

- **WandB Project Link:**
  https://wandb.ai/sushantak17-prom-iit-rajasthan/cifar10-cnn-lab2

10. Conclusion

A custom CNN was successfully trained on the CIFAR-10 dataset, achieving strong performance while maintaining reasonable computational complexity. Gradient flow

and weight update analyses provided insights into training stability. The results demonstrate effective model design and experiment tracking using MLOps tools.

11. GitHub Repository

The complete implementation is available at:

https://github.com/Sushantak17/MLOps-Sushantak-M25CSA035/tree/lab2_worksheet

---